UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

FACULTAD DE INGENIERÍA

LABORATORIO SISTEMAS OPERATIVOS 1



Manual de Usuario

Nombre	Carnet
Christopher Miguel Angel Ramos Ascencio	202200057

El Manual de Usuario proporciona instrucciones detalladas paso a paso para la inicialización, uso y monitoreo del sistema de gestión de contenedores utilizando métricas de kernel y la herramienta eBPF.

1.1 Requisitos y Entorno Previos

Para asegurar el correcto funcionamiento del sistema, la máquina host (preferiblemente una distribución basada en Debian/Ubuntu) debe cumplir con los siguientes requisitos:

1. **Docker y Docker Compose:** Requeridos para levantar los contenedores de prueba, la base de datos (Valkey) y el entorno de visualización (Grafana).
2. **Entorno de Go (Golang) 1.20+:** Necesario para compilar y ejecutar el demonio monitor.
3. **Dependencias del Kernel:** Herramientas de compilación (`build-essential`, `make`) y los headers del kernel (`linux-headers-$(uname -r)`).
4. **Permisos de Superusuario (root):** Requeridos para la inserción de módulos del kernel (`insmod`) y la ejecución de programas eBPF.

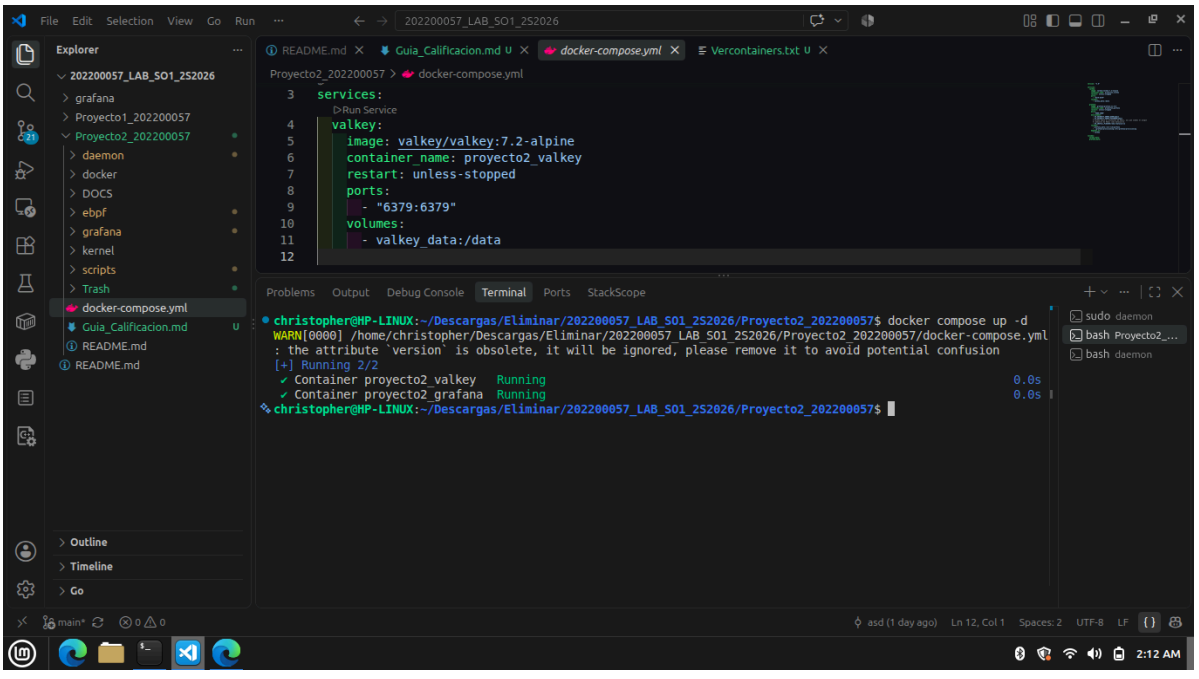
1.2 Proceso de Instalación y Despliegue

Paso 1: Despliegue de Infraestructura y Herramientas El sistema utiliza una arquitectura basada en microservicios manejada por Docker Compose. Ubíquese en el directorio raíz del proyecto y ejecute:

Bash
docker-compose up -d

Este comando inicializará:

- **Valkey (Redis-compatible):** Base de datos en memoria escuchando en el puerto 6379, encargada de almacenar las métricas procesadas.
- **Grafana:** Interfaz gráfica accesible en el puerto 3000 para el consumo de las métricas.



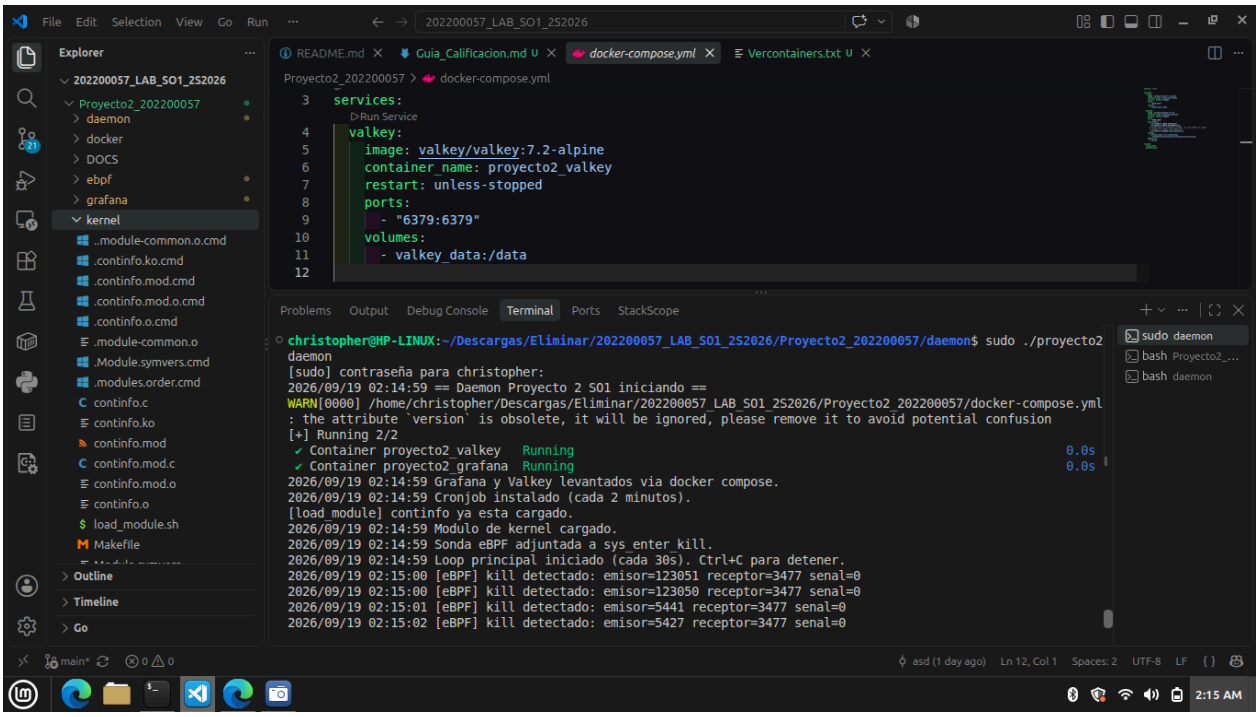
Paso 2: Compilación y Carga del Módulo de Kernel El sistema depende de un módulo escrito en C para extraer métricas precisas directamente de la tabla de procesos (`task_struct`). Navegue al directorio `/modulo` y ejecute:

Bash
make
sudo insmod continfo.ko

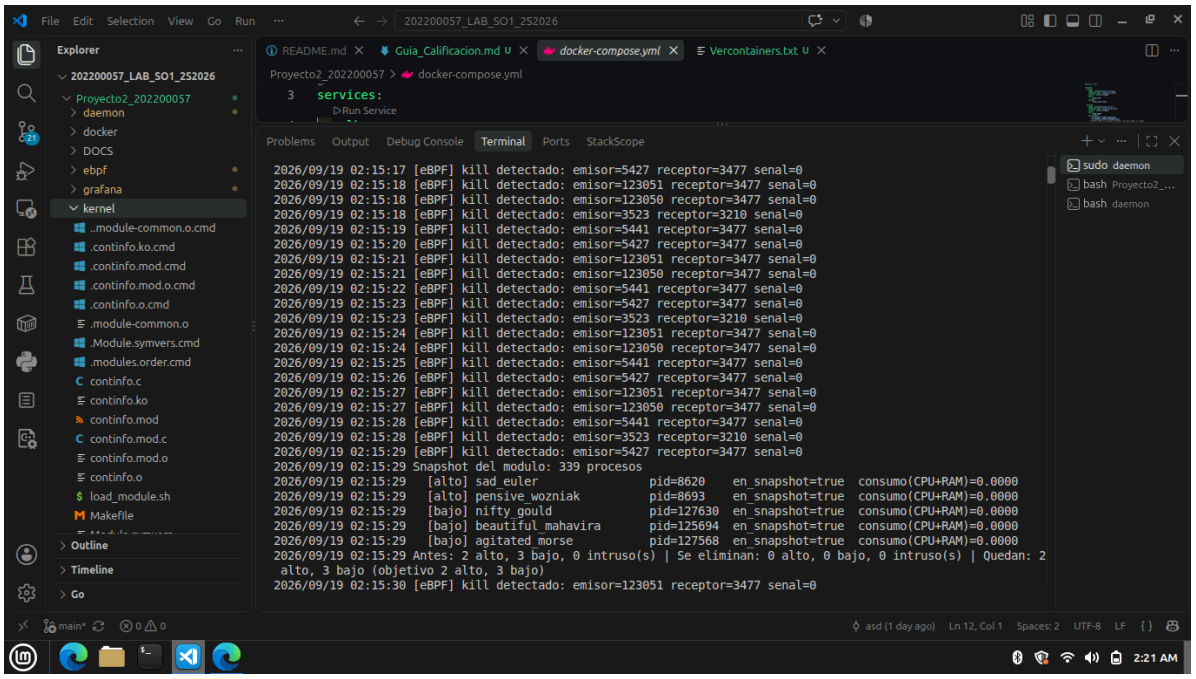
Comprobación: Puede verificar el correcto funcionamiento del módulo ejecutando `cat /proc/continfo_pr2_so1_202200057`. Debería observar un archivo JSON/texto con el estado de la RAM y los procesos.

Paso 3: Inicialización del Demonio Monitor El demonio escrito en Go orquestara todo el ciclo de vida del sistema, integrándose con Docker y eBPF. Navegue a la carpeta del demonio y compílelo/ejecútelo:

go build -o daemon .
sudo ./proyecto2daemon



esperar aproximadamente 2 minutos a que haya una vuelta completa para tener logs de esta manera:



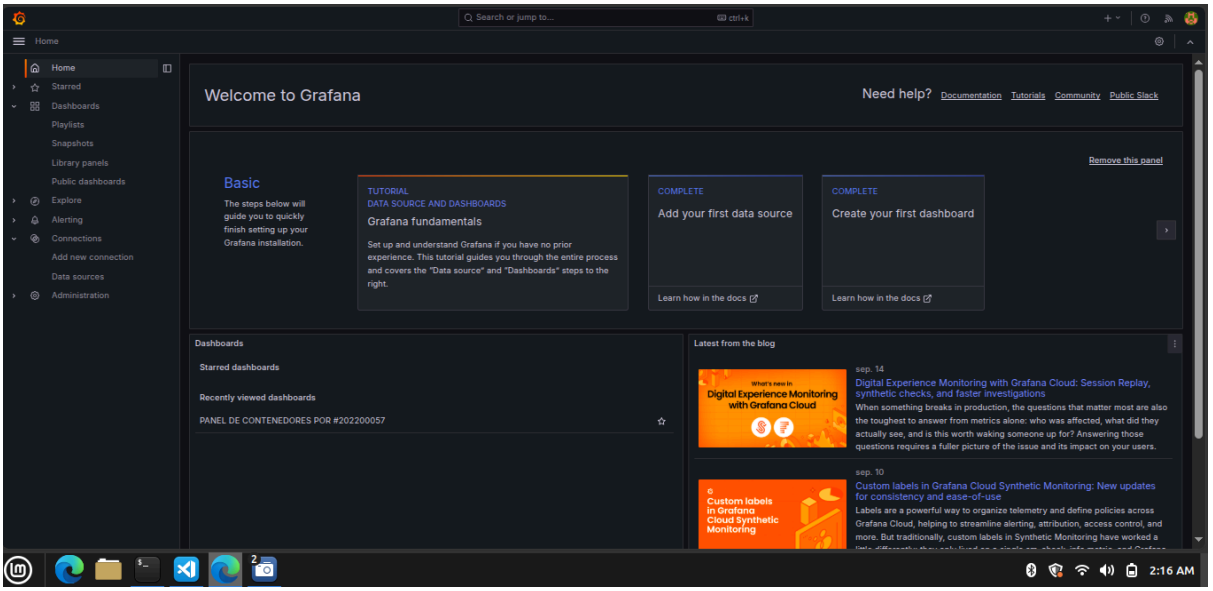
El demonio registrará tareas en el `cron` del sistema operativo, adjuntará las sondas eBPF y comenzará su bucle de monitoreo y balanceo de carga.

[Inserte captura de los logs del demonio indicando la correcta conexión a Docker, Valkey y eBPF]

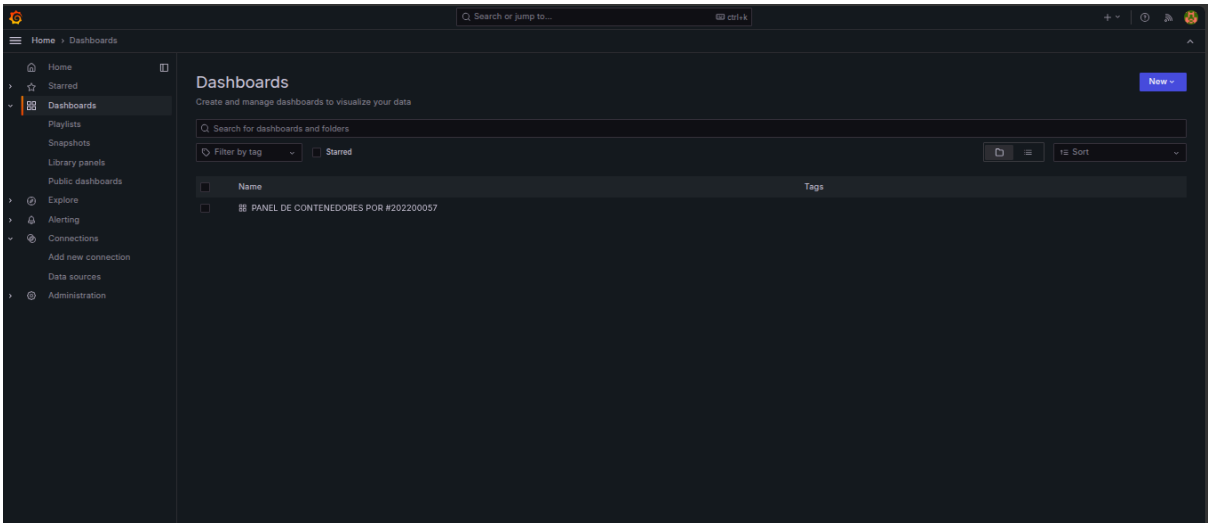
1.3 Operación y Navegación en Grafana

Con el demonio en ejecución, el flujo de datos hacia Valkey es continuo. Para visualizarlo:

1. Abra su navegador y diríjase a `http://localhost:3000`.
2. Las credenciales de acceso son las definidas en la configuración (por defecto `admin`/`admin`).



3. Navegue hacia **Dashboards** y seleccione el panel llamado **"PANEL DE CONTENEDORES POR #202200057"**.



Descripción de las Visualizaciones del Dashboard:

- **Métricas Globales (TOTAL RAM / FREE RAM):** Paneles de tipo `Stat` que muestran de forma inmediata el uso general de la memoria en megabytes.
- **Serie Temporal de RAM:** Gráfica `Timeseries` configurada mediante rangos de fechas (ZSET) que documenta el comportamiento de la RAM en el tiempo. Esta gráfica permite identificar anomalías (picos de consumo) generados por los contenedores de alto consumo.

- **Top 5 Procesos Demanadantes:** Paneles tipo `Piechart` que leen de la base de datos de manera interactiva mostrando qué procesos (PIDs) y nombres consumen la mayor cantidad de CPU y RAM.
- **Eventos de Terminación e Intercepción (eBPF):**
    - **TOTAL CONTENEDORES ELIMINADOS:** Contador absoluto que solo aumenta cuando el demonio logra eliminar un contenedor (Intruso o Excedente) y la eliminación es confirmada físicamente por la sonda eBPF.
    - **eBPF EVENT COUNTER:** Indicador crudo que cuenta cada señal (como `signal=0` o `signal=28`) interceptada en el sistema operativo, evidenciando el volumen de eventos capturados a nivel de kernel, sin importar si pertenecen a un contenedor.

En un momento



Actualizado en tiempo real



1.4 Apagado del Sistema

Para terminar la sesión de monitoreo garantizando que no existan fugas de recursos:

1. Detenga el demonio en la terminal presionando `Ctrl + C`. El sistema atrapará la señal y realizará un cierre limpio (eliminando el cron y desenlazando eBPF).
2. Detenga la infraestructura base y elimine el módulo del kernel:

Bash
docker-compose down
sudo rmmod continfo